Quick Practical Guide to Microbiome Sequencing

From 16S Reads to Composition with QIIME 2

Tomás Freire

tomas.fa.freire@gmail.com

1 Purpose

This document provides a quick and practical overview of how to process microbiome sequencing data using **QIIME2**. It is not exhaustive, but gives enough intuition to get started and understand the pipeline.

Goal:DNA reads $\longrightarrow$ sequence features $\longrightarrow$ taxonomic labels $\longrightarrow$ composition.

To get a first idea of how sequencing works, start with the Sanger sequencing video. For the microbiome, the extra complication is that we have a mixture of organisms: we get lots of sequences and then need to work out where they came from.

There are several sequencing methods:

- **Illumina**: short reads, often read from one or both ends of the fragment.
- **PacBio HiFi**: longer, accurate reads, which can cover the full 16S gene.
- **454 pyrosequencing**: an older method that you may find in published datasets.

The exact pipeline depends on the method and the region sequenced. The example commands here use `denoise-pyro`, for 454 data. For Illumina or PacBio, we would use a different denoising command. Check the dataset description to see which method was used.

2 What do the raw data look like?

A **read** is one DNA sequence that the machine has read. The raw data usually come in FASTQ files. Each read has four lines, for example:

```
@read_001
ACGTACGT
+
IIIIII!!
```

The first line is the read's name, the second is the DNA sequence, the third is a separator, and the fourth gives a quality score for each base. This is why the sequence and quality lines have the same length.

With Phred+33 encoding, `I` means $Q = 40$ and `!` means $Q = 0$. So, in this example, the last two bases are much less reliable than the others.

The quality score tells us how likely a base is to have been read incorrectly:

$$Q = -10 \log_{10}(p_{\text{error}}).$$

Quality score	Error probability per base	Interpretation
20	0.01	Approximately 1 in 100
30	0.001	Approximately 1 in 1,000
40	0.0001	Approximately 1 in 10,000

3 What is QIIME 2?

QIIME 2 is a bioinformatics platform for microbiome analysis. It brings together tools for quality control, denoising, classification and visualisation. It also keeps track of how each file was produced, which is useful when we want to check what we did.

- `.qza`: a data artifact, such as reads, a count table or taxonomy.
- `.qzv`: a visualisation, such as a quality plot or taxonomic bar plot.
- `.tsv`: a tab-separated text table, often used for sample metadata.

One of the nicest parts is that we can open the visualisations at https://view.qiime2.org/, or use `qiime tools view`. Keep the distinction in mind: the `.qza` contains the data, and the `.qzv` is usually what we open to look at the plots.

It can be a bit annoying to install. My suggestion is to use a separate environment, for example with Conda, and follow the instructions for your version. The commands below follow **QIIME 2 2024.10**; if you use another version, check the documentation or run the command with `-help`. You can find the current guide at https://amplicon-docs.qiime2.org/en/stable/.

There are two input files that are easy to confuse:

- **Manifest**: tells QIIME 2 where the sequencing files are and which sample each file belongs to. We use it for the import shown here, although other import formats are available.
- **Sample metadata**: tells us about the samples themselves, for example the participant, time point, treatment or Nugent score. The sample IDs need to match those in the sequencing data.

For the single-end Phred+33 V2 format used here, the manifest looks like this:

```
sample-id<TAB>absolute-filepath
S01<TAB>/absolute/path/S01.fastq.gz
S02<TAB>/absolute/path/S02.fastq.gz
```

The columns are separated by tabs: replace `<TAB>` with a real tab and change the paths to your own files. Paired-end data use a slightly different format.

4 Step by step: what is happening?

4.1 Import the data and look at the quality

First, we need to know which reads belong to which sample. This is called **demultiplexing** and uses sample-specific barcodes. If we already have a separate FASTQ file for each sample, this has already been done.

`demux_pyro.qza` contains the reads, separated by sample, and their quality scores. We then make `demux_pyro.qzv` to look at:

- how many reads we have per sample;
- how the quality changes along the sequence.

For example, the quality might be quite good until position 220 and then drop. We could cut the reads around that point to remove the least reliable part, but we would also lose some information.

4.2 Filtering and denoising

Filtering removes or trims low-quality reads. In the `q-score` example, we get a file with the filtered reads and `filter-stats_pyro.qza`, which tells us how many passed the quality check.

Denoising goes a bit further: it tries to distinguish real sequence differences from sequencing errors. DADA2 does this using an error model. It also filters reads and removes **chimeras**, which are sequences accidentally formed from parts of different templates during amplification.

Since DADA2 already does filtering, we do not necessarily need a separate `q-score` step first. In the code below, I show both, but DADA2 starts directly from `demux_pyro.qza`. If we want to use the prefiltered reads instead, we need to change that input and keep track of the extra step.

We get three important files:

- `table_pyro.qza`: how many reads support each feature in each sample.
- `rep-seqs_pyro.qza`: the DNA sequence representing each feature.
- `denoising-stats_pyro.qza`: how many reads survive the processing stages.

For example, we might start with 10,000 reads, keep 8,500 after filtering and finish with 8,000 after removing chimeras. It is useful to look at these numbers for each sample. If one group loses most of its reads and another does not, we should try to understand why before comparing them.

4.3 ASVs, OTUs and species

DADA2 gives us **amplicon sequence variants (ASVs)**. These are the different sequences that it infers for the region we sequenced. Two ASVs can differ by just one base.

These are the **representative sequences** stored in `rep-seqs_pyro.qza`: one DNA sequence for each ASV. For example, if 600 reads in a sample are assigned to ASV1, we do not need to classify all 600 separately. We classify the sequence representing ASV1, while the count table keeps track of how many reads were assigned to it in each sample.

At this point, we have the sequences and their counts, but we have not yet assigned organism names to them. The next step is to classify these representative sequences using a reference database.

You may also see **operational taxonomic units (OTUs)** in papers. Here, sequences are grouped by similarity, often using a threshold of 97%. This refers to the DNA sequences: it does not mean that an OTU is always one species.

Important: an ASV is not necessarily a species. One species can give us several ASVs, for example because its strains differ or because it has slightly different copies of the 16S gene. The opposite can also happen: two species can have the same sequence in the region we measured, so we cannot tell them apart from that sequence alone.

4.4 Assign taxonomy: which organisms do the sequences belong to?

Now we take the representative sequences and run a classifier to assign taxonomy. Databases such as **Greengenes2** and **SILVA** contain reference sequences with known labels. A pretrained classifier has already learnt from a reference database, so we can use it without training one ourselves.

The result is `taxonomy.qza`. Some sequences may get a species name; others may only get a genus or family. That is fine: the sequence does not always contain enough information to go further.

Different databases can also use different names for the same organism. Check which version you are using and whether the classifier matches your QIIME 2/scikit-learn version and the region sequenced. A classifier trained on full-length sequences is not automatically the best option for short reads. The confidence score is useful, but does not guarantee that a species label is correct.

4.5 Manual taxonomy: looking at sequences with BLAST

Sometimes a sequence is not classified as far as we would like. We can export the representative sequences as FASTA, take one of them and look it up using NCBI nucleotide BLAST. We are checking its classification here, rather than sequencing it again.

Two useful things to look at are:

- **Query coverage**: how much of our sequence is covered by the match?
- **Identity**: how similar are the sequences in the part that matches?

Also look at the length, the quality of the reference and whether the other good matches point to the same organism.

5 End result: bar plots and a composition table

Once we have the taxonomy, we can combine sequences with the same label and make bar plots. We choose the level we want to work at, for example genus or species, depending on what the data can support.

In the example above, adding ASV1 and ASV2 gives us 800 *L. crispatus* reads in S01 and 200 in S02.

To turn counts into relative abundances, we divide by the total number of retained reads in each sample. If c_{sj} is the count for taxon j in sample s,

$$z_{sj} = \frac{c_{sj}}{\sum_k c_{sk}}, \qquad \sum_j z_{sj} = 1.$$

Both samples in our example have 1,000 reads, so we get:

Sample	*L. crispatus*	*L. iners*	Unclassified
S01	0.80	0.10	0.10
S02	0.20	0.50	0.30

This is the table we can use for modelling: one row per sample, one column per taxon, and each row sums to one. We can then add things like the Nugent score or treatment using the sample IDs.

A A quick walkthrough of the notebook

The notebook takes us from the raw sequencing files to a taxonomic bar plot. Here are the main steps, with a short explanation of what each one does. These commands are written for a terminal; in a notebook, add `!` before each command.

A.1 Prepare the files

First, download the sequencing files from SRA022855 and put them in the same folder. We also need:

- `manifest.tsv`: sample IDs and paths to the sequencing files;
- `sample-metadata.tsv`: information about each sample.

The Python code at the start of the notebook creates a basic metadata file from the filenames. The `Unknown` entries are placeholders that we can replace with real information. It does not create the manifest, so that needs to be prepared separately. Make sure the sample IDs match in both files.

We can open the first two reads to see what the raw data look like:

```
gunzip -c SRR062670.fastq.gz | head -n 8
```

This shows the sequences and quality characters. The import below assumes Phred+33 encoding; confirm this from the data source, since looking at a few reads is not always enough to establish the encoding.

A.2 Import the reads and look at the quality

```
qiime tools import \
  --type 'SampleData[SequencesWithQuality]' \
  --input-path manifest.tsv \
  --input-format SingleEndFastqManifestPhred33V2 \
  --output-path demux_pyro.qza

qiime demux summarize \
  --i-data demux_pyro.qza \
  --o-visualization demux_pyro.qzv
```

Open `demux_pyro.qzv` at https://view.qiime2.org/. This lets us see how many reads we have per sample and how their quality changes along the sequence.

A.3 Filter and denoise

In this notebook, we first filter the reads and then pass the filtered file to DADA2:

```
qiime quality-filter q-score \
  --i-demux demux_pyro.qza \
  --o-filtered-sequences demux-filtered_pyro.qza \
  --o-filter-stats filter-stats_pyro.qza

qiime dada2 denoise-pyro \
  --i-demultiplexed-seqs demux-filtered_pyro.qza \
  --p-trunc-len 200 \
  --o-table table_pyro.qza \
  --o-representative-sequences rep-seqs_pyro.qza \
  --o-denoising-stats denoising-stats_pyro.qza
```

The value 200 is the truncation length used here; for another dataset, we should choose it from the quality plots and the region sequenced. The `denoise-pyro` command is for 454/Ion Torrent reads, rather than a general command for every sequencing method.

The main outputs are:

- `table_pyro.qza`: ASV counts in each sample;
- `rep-seqs_pyro.qza`: one representative DNA sequence per ASV;
- `denoising-stats_pyro.qza`: how many reads passed each stage.

A.4 Assign taxonomy with Greengenes2

The notebook downloads a pretrained Greengenes2 classifier. After downloading it, we run:

```
qiime feature-classifier classify-sklearn \
  --i-classifier 2024.09.backbone.full-length.nb.sklearn-1.4.2.qza \
  --i-reads rep-seqs_pyro.qza \
  --o-classification taxonomy.qza
```

This assigns taxonomic labels to the representative sequences. Check that the downloaded classifier is compatible with the installed QIIME 2/scikit-learn version and suitable for the region sequenced.

A.5 Check some assignments manually

Some sequences may only be classified to genus level. We can look at selected sequences with NCBI nucleotide BLAST to see whether a more detailed assignment is supported. Sometimes it is; sometimes the sequence matches several species equally well.

If we make changes, keep the original taxonomy and save a separate table as `taxonomy_manual.tsv`. The import below expects a tab-separated file **without a header**, with a feature ID and its taxonomic assignment on each row. Keep the feature IDs unchanged and make a note of the assignments we changed and why.

```
qiime tools import \
  --type 'FeatureData[Taxonomy]' \
  --input-format HeaderlessTSVTaxonomyFormat \
  --input-path taxonomy_manual.tsv \
  --output-path manual-taxonomy_pyro.qza
```

A.6 Make the bar plots

Finally, we combine the ASV counts, taxonomy and sample metadata:

```
qiime taxa barplot \
  --i-table table_pyro.qza \
  --i-taxonomy manual-taxonomy_pyro.qza \
  --m-metadata-file sample-metadata.tsv \
  --o-visualization taxa-barplot-GG2-manual_pyro.qzv
```

Open the result at https://view.qiime2.org/ to explore the composition of each sample at different taxonomic levels. If we did not make manual changes, use `taxonomy.qza` instead.

The notebook ends at the bar plot. To use the composition in a model, we would then export a table at the taxonomic level we want and arrange it with samples as rows and taxa as columns.